

Informe T2

Nicolas Sanchez

Repositorio Git: https://github.com/nico031022/t2_Procesamiento_Imagenes_nicolas_sanchez.git

1. P1

1.1. Objetivo

En esta pregunta busco entender cómo ajustar el valor adecuado de σ en un filtro Gaussiano cuando el nivel de ruido no es constante, sino que depende de la intensidad de la imagen.

Para evaluarlo, armé una imagen sintética con tres niveles de intensidad, agregué ruido Poisson y fui probando distintos valores de σ para ver cuál entregaba el menor RMSE en cada zona. Con estos resultados en mente, diseñé un filtro adaptativo que ajusta su σ espacialmente según una estimación local de la intensidad

1.2. Imagen sintética y ruido Poisson

Generé una imagen de 256×256 píxeles dividida en tres regiones:

- Background: intensidad de 0,15.
- Square central de 128×128 píxeles: intensidad de 0,45.
- Circle de radio 32 píxeles: intensidad de 0,80.

También creé tres máscaras para calcular los errores por separado en cada zona. Decidí mantener los píxeles cercanos a las fronteras dentro de las máscaras, ya que el error de desenfoque que aparece al suavizar los bordes también es parte del RMSE real que me interesa medir.

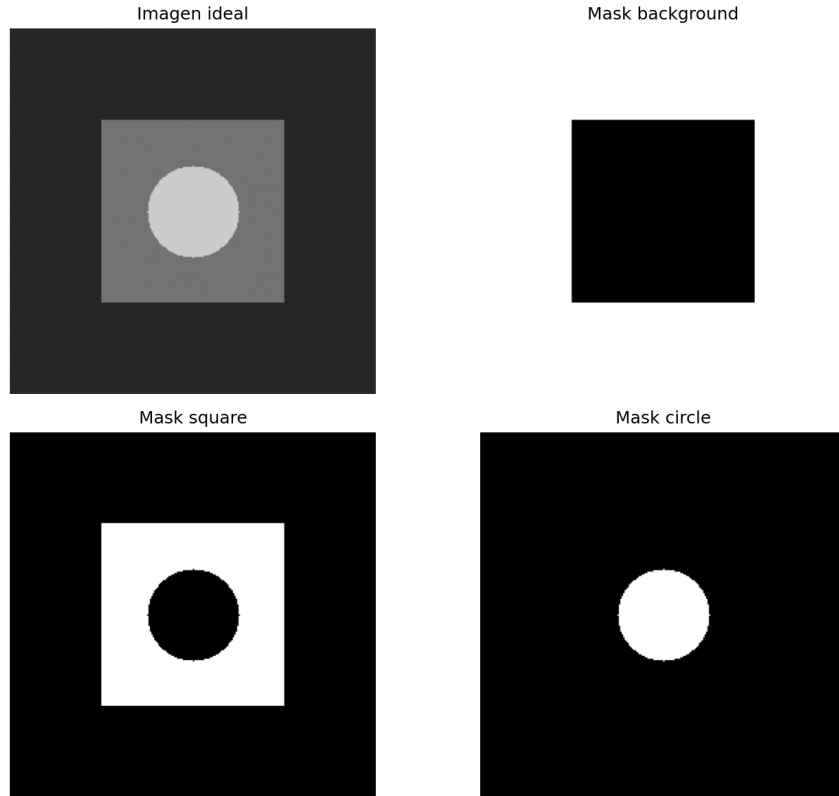


Figura 1: Imagen sintética de 256×256 y máscaras usadas para background, square y circle.

Para simular el ruido, generé la imagen ruidosa usando la relación:

$$y = \frac{\text{Poisson}(Nx)}{N}, \quad N = 40,$$

fijando una semilla aleatoria para poder replicar el experimento exactamente igual.

Las medias que obtuve fueron 0,14980, 0,44946 y 0,79867 para el background, square y circle, respectivamente. Son valores muy cercanos a las intensidades originales. Por otro lado, las varianzas fueron 0,00373, 0,01147 y 0,01951, lo que comprueba empíricamente que la varianza crece a medida que aumenta la intensidad.

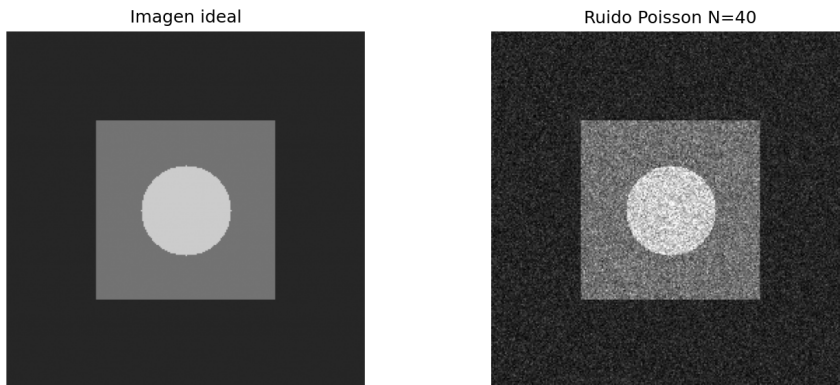


Figura 2: Imagen ideal e imagen con ruido Poisson usando $N = 40$.

1.3. Filtro Gaussiano y búsqueda de σ

Armé el clásico kernel Gaussiano isotrópico:

$$G(i, j) = \exp\left(-\frac{i^2 + j^2}{2\sigma^2}\right).$$

Como en teoría la campana de Gauss es infinita, tuve que cortarla usando un radio finito:

$$r = \lceil 3\sigma \rceil,$$

asi que el kernel me queda de tamaño $(2r + 1) \times (2r + 1)$. Elegí 3σ porque fuera de ese rango los valores son tan chicos que su aporte es prácticamente nulo.

Después normalicé el kernel para cumplir que:

$$\sum_{i,j} G(i, j) = 1.$$

Hice esto para asegurarme de no alterar artificialmente el brillo de una zona constante por errores numéricos al sumar los coeficientes. Como prueba inicial usé $\sigma = 1,3$, obteniendo un kernel de 9×9 cuya suma fue igual a 1.

Y para buscar el σ ideal, hice un barrido probando valores:

$$\sigma = 0,1,0,2, \dots, 4,0,$$

además de medir el caso base sin filtrado ($\sigma = 0$). Para cada caso, calculé el RMSE global y el de las tres regiones comparándolo con la imagen ideal limpia:

$$RMSE = \sqrt{\frac{1}{n} \sum_{k=1}^n (x_k - \hat{x}_k)^2}.$$

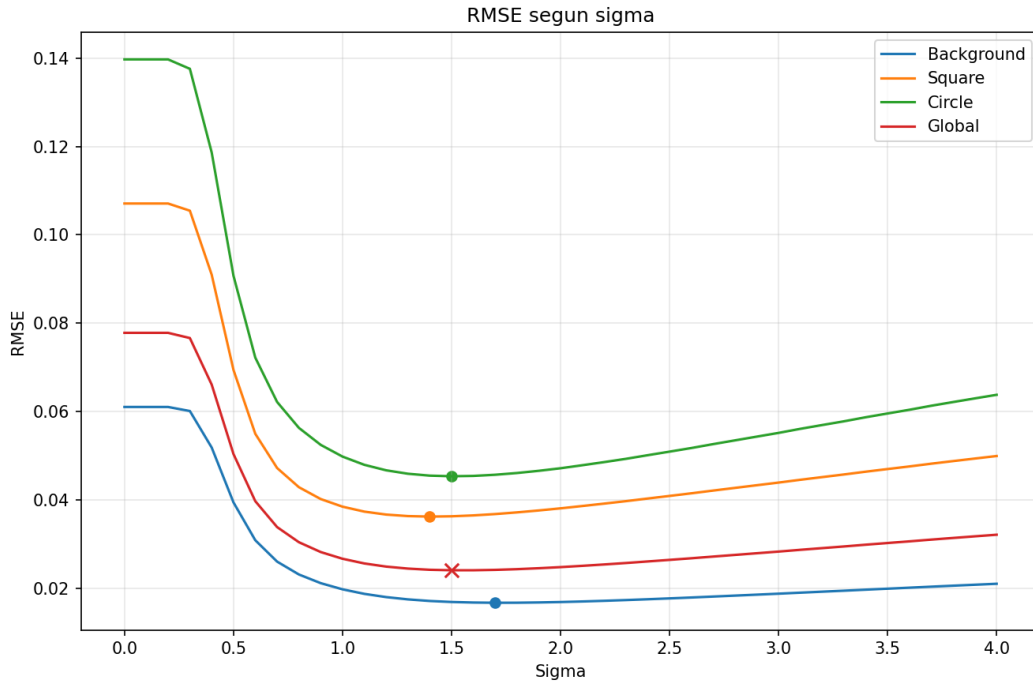


Figura 3: RMSE para $N = 40$, incluyendo el caso sin filtro y el barrido $\sigma = 0,1$ a $4,0$ con paso $0,1$.

Los mínimos que encontré fueron:

Región	σ^*	RMSE mínimo
Global	1.5	0.024085
Background	1.7	0.016730
Square	1.4	0.036231
Circle	1.5	0.045359

Cuadro 1: Valores de σ que minimizan el RMSE en cada región.

Como era de esperarse, el mismo σ no resulta óptimo para todas las zonas. La intensidad importa porque determina la cantidad de ruido, pero esto también puede influir cuánto error introduzco al difuminar una frontera.

El background tiene muchos píxeles lejos de los bordes, por lo que tolera más suavizado y su óptimo fue $\sigma = 1,7$. El square, en cambio, limita tanto con el background como con el circle, así que el error introducido al suavizar los bordes tiene más peso y su mejor valor bajó a 1,4. Por último, el circle tiene el ruido más alto, pero como solo limita con el square, el compromiso entre reducir el ruido y conservar el borde lo dejó en $\sigma = 1,5$.

1.4. Diseño del filtro adaptativo

Para que el filtro decida qué σ usar dependiendo de dónde esté, primero saqué una estimación de la intensidad local $\hat{\mu}(x, y)$ pasándole un Gaussiano auxiliar a la imagen ruidosa:

$$\hat{\mu} = G_{\sigma_\mu} * y, \quad \sigma_\mu = 2,0.$$

Elegí $\sigma_\mu = 2,0$ como un valor intermedio: reduce las variaciones pequeñas producidas por el ruido, pero sin suavizar demasiado las transiciones entre las regiones. Las intensidades promedio que estimé fueron 0,15220, 0,44478 y 0,78107.

Para armar la función $\sigma = F(\hat{\mu})$, interpolé linealmente basándome en las intensidades nominales de la imagen y los σ óptimos que saqué en el barrido:

$$(0,15, 1,7), \quad (0,45, 1,4), \quad (0,80, 1,5).$$

Si la intensidad cae por debajo de 0,15 lo dejé fijo en $\sigma = 1,7$, y si pasa de 0,80 lo dejé en $\sigma = 1,5$.

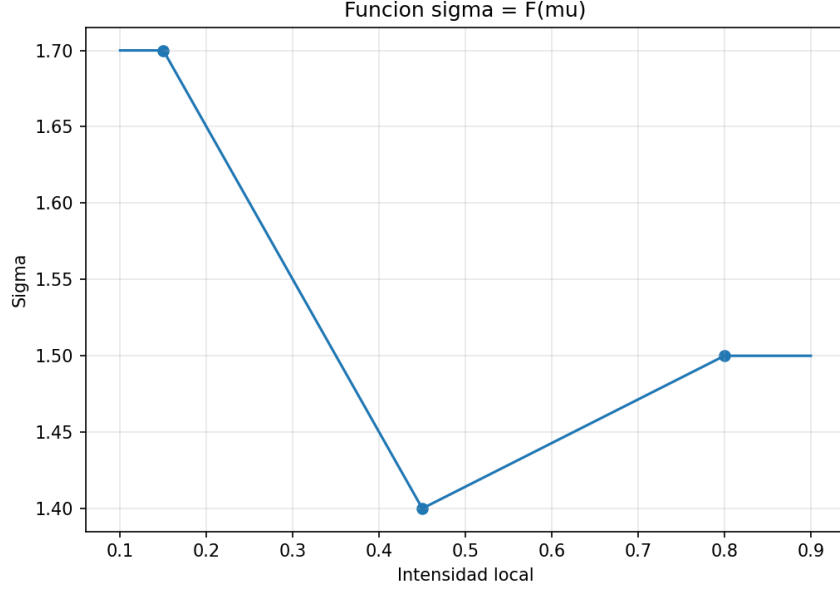


Figura 4: Función $\sigma = F(\hat{\mu})$ obtenida por interpolación de los tres valores óptimos regionales.

Al aplicar esta función a toda la imagen, obtuve el mapa de σ .

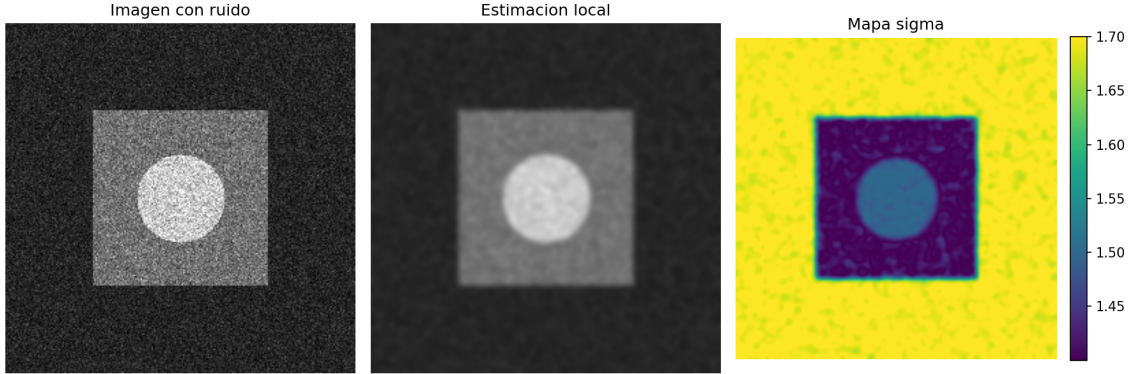


Figura 5: Imagen ruidosa, estimación $\hat{\mu}$ con $\sigma_{\mu} = 2,0$ y mapa de σ resultante.

Los promedios que me dio el mapa fueron 1,6943 en el background, 1,4166 en el square y 1,4930 en el circle, bastante consistentes con lo que arrojó el barrido.

A nivel de código, para no complicar tanto la implementación del filtro adaptativo, usé un tamaño de kernel fijo para todos los píxeles calculado en base al mayor valor del mapa. Como $\sigma_{\text{máx}} = 1,7$, se obtiene:

$$r_{\text{máx}} = \lceil 3(1,7) \rceil = 6,$$

por lo que todos los kernels son de 13×13 . Aunque la matriz sea del mismo tamaño, los coeficientes cambian según el σ local y vuelvo a normalizar cada kernel de forma individual.

1.5. Comparación entre filtro global y adaptativo

El mejor filtro Gaussiano global de mi barrido fue el de $\sigma = 1,5$. Al comparar los resultados:

Método	Global	Background	Square	Circle
Sin filtro	0.077812	0.061039	0.107076	0.139686
Gaussiano global	0.024085	0.016909	0.036299	0.045359
Gaussiano adaptativo	0.023959	0.016459	0.036737	0.045085

Cuadro 2: RMSE del caso sin filtro, Gaussiano global con $\sigma = 1,5$ y Gaussiano adaptativo con $\sigma_\mu = 2,0$.

Al comparar los valores, el filtro adaptativo logra reducir ligeramente el RMSE global (de 0,024085 a 0,023959). La mejora es marginal, pero existe.

Donde más se nota es en el background, ya que ahora puede usar libremente su valor ideal de 1,7. En el circle la diferencia es mínima porque su óptimo ya era 1,5 (igual que el global).

En el square, en cambio, el RMSE subió un poco (de 0,036299 a 0,036737). Creo que esto puede deberse a los píxeles que están justo en la frontera: ahí la estimación $\hat{\mu}$ toma valores intermedios y por ende σ no se queda exactamente en 1,4. Por ejemplo, cerca del borde background-square obtuve $\sigma(60) = 1,6803$, $\sigma(64) = 1,5119$ y $\sigma(68) = 1,4100$. Esto hace que algunos píxeles de la orilla se suavicen un poco más que los del interior del square.

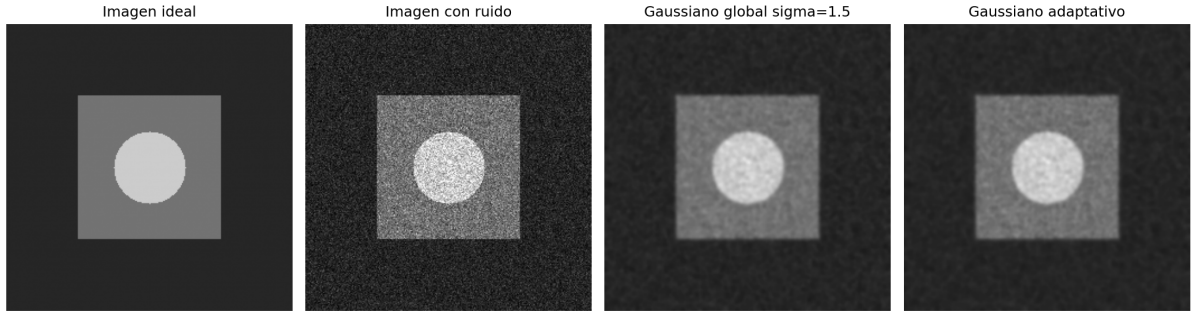


Figura 6: Comparación entre la imagen ideal, la imagen ruidosa, el Gaussiano global con $\sigma = 1,5$ y el filtro adaptativo.

1.6. Comportamiento cerca de las fronteras

Para ver en detalle qué pasa en las transiciones, saqué un perfil horizontal justo por el medio de la imagen, cruzando los saltos:

$$0,15 \rightarrow 0,45 \rightarrow 0,80 \rightarrow 0,45 \rightarrow 0,15.$$

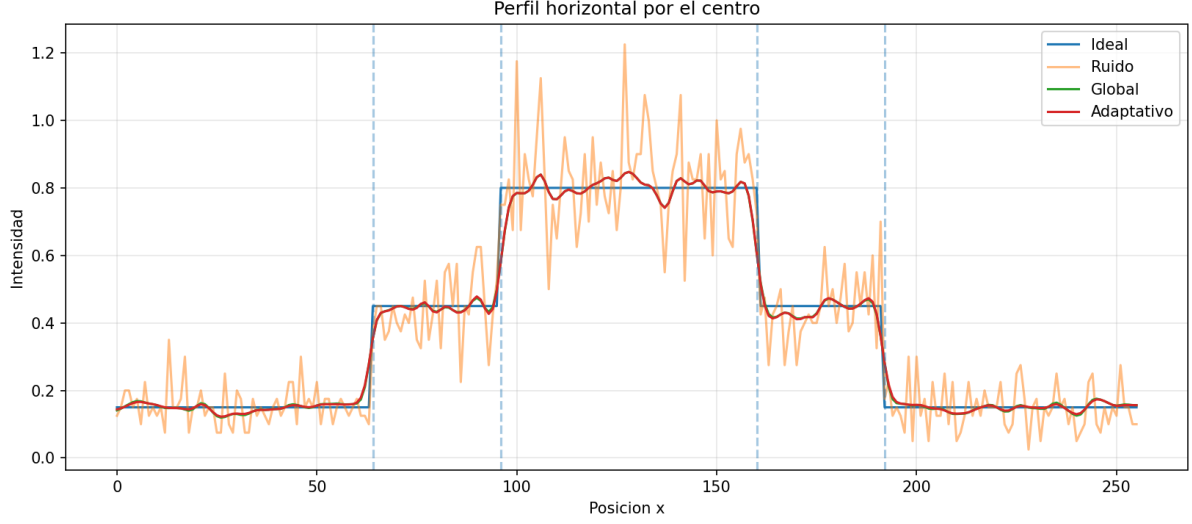


Figura 7: Perfil horizontal central para el Gaussiano global ($\sigma = 1,5$) y el adaptativo ($\sigma_\mu = 2,0$).

Se observa que ambos filtros suavizan los saltos bruscos de intensidad, lo que es esperable porque en ambos casos se siguen utilizando filtros Gaussianos. No observé discontinuidades adicionales ni halos visibles al variar σ espacialmente.

El perfil del mapa de σ también me confirma que la transición es más bien gradual.

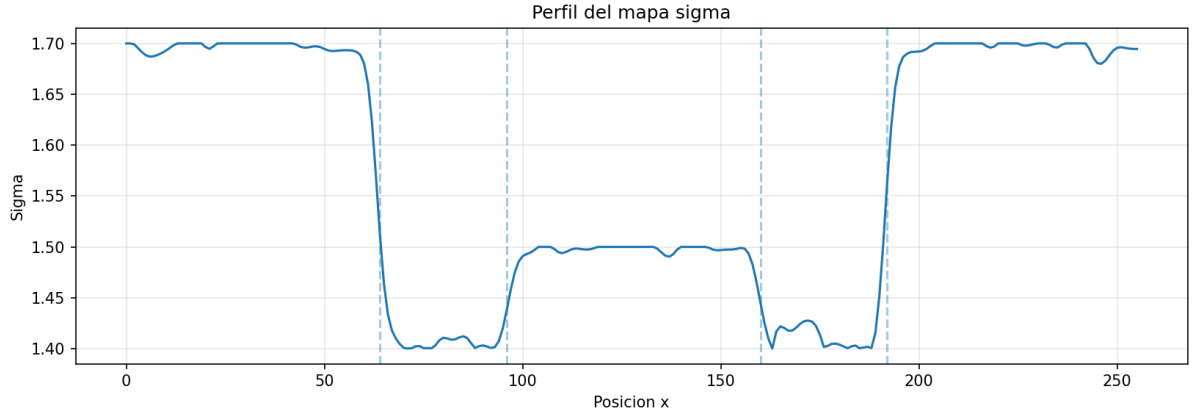


Figura 8: Perfil horizontal del mapa de σ , obtenido usando $\sigma_\mu = 2,0$.

En la frontera background-square medí:

$$\sigma(60) = 1,6803, \quad \sigma(64) = 1,5119, \quad \sigma(68) = 1,4100,$$

y en la frontera square-circle:

$$\sigma(92) = 1,4007, \quad \sigma(96) = 1,4381, \quad \sigma(100) = 1,4908.$$

El mapa no presenta cambios abruptos. De todas formas, el filtro es isotrópico, por lo que cerca de una frontera todavía mezcla información de ambos lados. Una modificación simple sería suavizar el mapa de σ antes de usarlo o utilizar un σ_μ un poco mayor para la estimación.

1.7. Píxeles representativos

Elegí un píxel representativo, bien alejado de los bordes en cada región, para ver qué valores reales estaba usando el filtro adaptativo.

Región	Píxel	$\hat{\mu}$	σ	Kernel
Background	(32, 32)	0.15515	1.69485	13×13
Square	(80, 80)	0.44540	1.40460	13×13
Circle	(128, 128)	0.83537	1.50000	13×13

Cuadro 3: Valores usados por el filtro en tres píxeles representativos.

Aunque los tres usan una matriz de 13×13 , se nota cómo cambia la forma. El kernel del background es más ancho y extendido porque tiene el σ más alto, mientras que el del square está más concentrado en el centro. El del circle presenta un comportamiento intermedio.

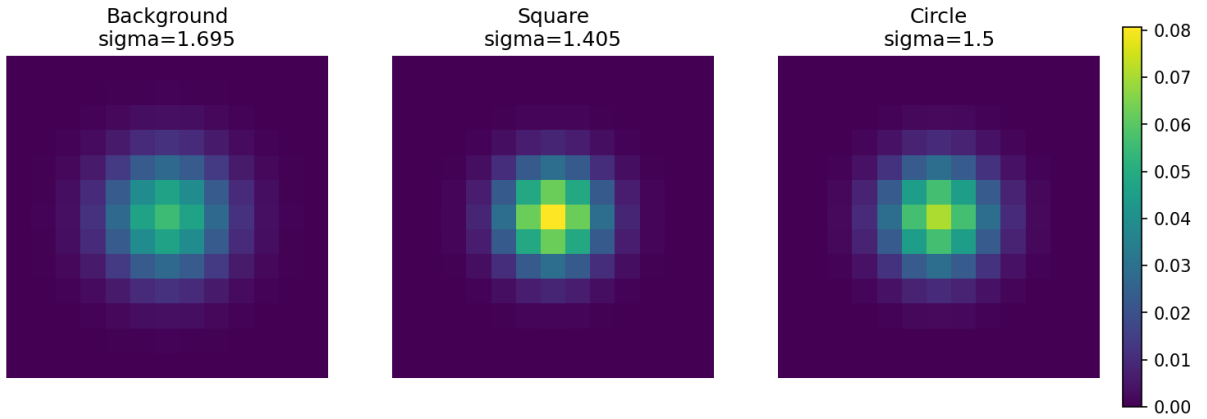


Figura 9: Kernels locales usados en los tres píxeles representativos.

2. Respuestas a las preguntas guiadas

2.1. ¿Por qué la varianza del ruido Poisson depende de la intensidad?

Por definición matemática, si $Z \sim \text{Poisson}(\lambda)$, entonces su media y su varianza son iguales a λ . En mi caso apliqué:

$$y = \frac{\text{Poisson}(Nx)}{N},$$

asi que sacando esperanza y varianza me queda:

$$E[y] = x, \quad \text{Var}(y) = \frac{x}{N}.$$

Entonces, manteniendo N fijo, una región de mayor intensidad (x) inevitablemente tendrá mayor varianza de ruido. Esto coincide con lo que medí empíricamente: 0,00373, 0,01147 y 0,01951 al ir subiendo la intensidad.

En las curvas de RMSE se observa con claridad que, sin filtrar, el circle parte con el error más alto de todos. Al subir el σ elimino ese ruido, pero introduzco error por desenfocar los bordes. Por eso la intensidad cambia las curvas, aunque no es lo único que importa para elegir el σ final.

2.2. ¿Por qué se incluyen píxeles cercanos a los bordes?

La razón principal es que, si mido solo el centro plano de la figura, estaría ignorando una parte importante del problema. Al aplicar cualquier filtro espacial, los bordes siempre se van a difuminar un poco, y ese error hay que considerarlo en el desempeño real del filtro.

Si midiera solo una zona plana lejos de cualquier borde, el error por desenfoque desaparecería y el σ ideal podría resultar mucho mayor, ya que el filtro solo buscaría reducir el ruido sin considerar la pérdida de las fronteras.

2.3. ¿Cómo se determina el tamaño del kernel y por qué se normaliza?

El tamaño lo definí cortando la función en $r = \lceil 3\sigma \rceil$, lo que da un tamaño total de $(2r + 1) \times (2r + 1)$. Hago esto porque la campana de Gauss ahí ya decae a valores casi nulos, así que no vale la pena seguir calculando píxeles infinitos.

Lo normalicé asegurando que $\sum_{i,j} G(i, j) = 1$ para no aumentar ni reducir artificialmente la intensidad de la imagen. Si no hiciera esto, al filtrar una zona plana podría cambiar el brillo debido únicamente a la suma de los coeficientes.

2.4. ¿Cómo se obtiene $\hat{\mu}(x, y)$ y cómo se transforma en $\sigma(x, y)$?

La estimación la saqué pasándole un filtro Gaussiano inicial con $\sigma_\mu = 2,0$ a la imagen ruidosa para tener una idea base de la intensidad. Luego, transformé esa intensidad en $\sigma(x, y)$ metiendo el valor en una función por tramos (interpolación lineal) basada en los óptimos que encontré: (0,15, 1,7), (0,45, 1,4) y (0,80, 1,5).

Por ejemplo, para mis tres píxeles de prueba me dio:

Región	Píxel	$\hat{\mu}$	σ
Background	(32, 32)	0.15515	1.69485
Square	(80, 80)	0.44540	1.40460
Circle	(128, 128)	0.83537	1.50000

En los tres usé el mismo soporte de 13×13 , pero el peso de los coeficientes cambia según el σ de la tabla.

2.5. ¿Por qué el filtro adaptativo no puede escribirse como una única convolución con un kernel fijo?

Porque una convolución clásica usa el mismo kernel en todas partes, es decir, es espacialmente invariante. En este filtro adaptativo, el kernel cambia píxel a píxel dependiendo del valor indicado por el mapa de $\sigma(x, y)$. Como el comportamiento del filtro depende de la posición, no se puede representar como una única convolución estándar con un kernel fijo.

Una aproximación posible sería discretizar el mapa de σ en unos pocos valores, filtrar la imagen con esos kernels fijos y luego seleccionar o interpolar los resultados según el σ asignado a cada píxel.

2.6. Comparación final entre el mejor Gaussiano global y el adaptativo

El adaptativo gana por un margen pequeño al global (0,023959 versus 0,024085).

La mejora se nota principalmente en el background (de 0,016909 a 0,016459). En el circle el resultado queda casi igual, con una mejora muy pequeña, porque su óptimo ya era cercano a 1,5. En el square, en cambio, el RMSE aumenta ligeramente. Creo que esta pequeña pérdida puede deberse a que cerca de las fronteras el mapa toma valores intermedios de σ , produciendo un poco más de suavizado en las orillas del cuadrado.

En conclusión, adaptar el σ según la intensidad sí ayudó a mejorar el RMSE global, pero el compromiso entre reducir el ruido y mantener los bordes nítidos sigue siendo importante.

3. P2

En esta pregunta logré implementar parte de la estructura necesaria para trabajar con difusión anisotrópica. Programé las diferencias finitas en cuatro direcciones, el cálculo del gradiente y del Laplaciano, el coeficiente de Variación Total (TV) y una primera versión del esquema explícito para el caso TV.

Para revisar que estas funciones estuvieran operando correctamente, utilicé una imagen sintética simple. Debido a la falta de tiempo, no alcancé a completar las otras variantes ni todos los experimentos pedidos, por lo que esta sección se enfoca principalmente en las pruebas básicas de mi implementación.

3.1. Imagen de prueba

Utilicé una imagen sintética de 20×20 píxeles, con un fondo de intensidad 0 y un cuadrado central de intensidad 1. Mi idea fue trabajar primero con una imagen sencilla, donde me fuera fácil observar el comportamiento de las funciones tanto en las zonas homogéneas como cerca de los bordes.

3.2. Diferencias finitas

Para la difusión, calculé las diferencias entre cada píxel y sus vecinos en las cuatro direcciones:

$$d_N = u_{i-1,j} - u_{i,j} \quad (1)$$

$$d_S = u_{i+1,j} - u_{i,j} \quad (2)$$

$$d_L = u_{i,j-1} - u_{i,j} \quad (3)$$

$$d_R = u_{i,j+1} - u_{i,j} \quad (4)$$

Para manejar los bordes apliqué un padding tipo **edge**, replicando el valor más cercano hacia afuera de la imagen. Al aplicar estas diferencias sobre la imagen de prueba, obtuve valores entre -1 y 1. Como era de esperarse, los valores distintos de cero aparecen casi exclusivamente alrededor del borde del cuadrado, justo donde ocurre el cambio brusco entre las intensidades 0 y 1.

3.3. Gradiente y Laplaciano

A partir de las diferencias anteriores, aproximé las derivadas horizontal y vertical de la siguiente manera:

$$g_x = \frac{d_R - d_L}{2} \quad (5)$$

$$g_y = \frac{d_S - d_N}{2} \quad (6)$$

Luego, calculé la magnitud del gradiente mediante:

$$|\nabla u| = \sqrt{g_x^2 + g_y^2} \quad (7)$$

Para la imagen de prueba obtuve los siguientes extremos:

$$|\nabla u|_{\min} = 0, \quad |\nabla u|_{\max} \approx 0,7071 \quad (8)$$

Tal como esperaba, la mayor respuesta del gradiente aparece justo en los bordes del cuadrado, mientras que en las zonas planas da cero.

Por otro lado, calculé el Laplaciano sumando las cuatro diferencias direccionales:

$$\nabla^2 u = d_N + d_S + d_L + d_R \quad (9)$$

Aquí obtuve valores en el rango:

$$\nabla^2 u_{\min} = -2, \quad \nabla^2 u_{\max} = 1 \quad (10)$$

El Laplaciano marca valores positivos y negativos justo en las transiciones de intensidad. Dejé esta función implementada como base para las variantes posteriores de difusión.

3.4. Coeficiente de difusión TV

Como primer coeficiente, implementé la aproximación regularizada de Variación Total indicada en el enunciado:

$$c_{TV} = \frac{1}{\sqrt{|\nabla u|^2 + \epsilon^2}} \quad (11)$$

Dejé el parámetro ϵ como entrada de la función para poder cambiarlo entre distintas pruebas. Para una primera pasada, fijé:

$$\epsilon = 0,1 \quad (12)$$

Obteniendo así:

$$c_{\min} \approx 1,4003, \quad c_{\max} = 10 \quad (13)$$

En una zona plana, donde $|\nabla u| \approx 0$, el coeficiente tiende aproximadamente a:

$$c_{TV} \approx \frac{1}{\epsilon} \quad (14)$$

Por esta razón, para $\epsilon = 0,1$ el valor máximo posible del coeficiente es 10.

En los bordes, como el gradiente aumenta, el valor de c disminuye. Esto permite una mayor difusión en las zonas homogéneas y reduce el suavizado cerca de los cambios fuertes de intensidad.

3.5. Difusión anisotrópica para el caso TV

Después, armé la actualización explícita utilizando las diferencias en las cuatro direcciones:

$$u^{t+1} = u^t + \lambda (c_N d_N + c_S d_S + c_L d_L + c_R d_R) \quad (15)$$

En cada iteración, vuelvo a calcular las diferencias y el coeficiente TV utilizando la imagen actualizada.

Obtuve los coeficientes direccionales promediando el valor de c de mi píxel actual con el de su vecino respectivo. Por ejemplo, para la dirección norte:

$$c_N = \frac{c_{i,j} + c_{i-1,j}}{2} \quad (16)$$

Apliqué exactamente la misma lógica para las otras tres direcciones.

3.5.1. Paso temporal

Para escoger el paso temporal (λ), me basé en el criterio:

$$\lambda \leq \frac{1}{4c_{\text{máx}}} \quad (17)$$

Dado que en el caso TV se cumple:

$$\frac{1}{\sqrt{|\nabla u|^2 + \epsilon^2}} \leq \frac{1}{\epsilon} \quad (18)$$

Entonces, para $\epsilon = 0,1$ tenemos que:

$$c_{\text{máx}} \leq 10 \quad (19)$$

Y el límite seguro para mi paso temporal queda en:

$$\lambda \leq \frac{1}{4(10)} = 0,025 \quad (20)$$

En mis pruebas decidí usar:

$$\lambda = 0,02 \quad (21)$$

Asegurándome de quedar bien bajo este límite teórico. Al correr 10 iteraciones con $\epsilon = 0,1$ y $\lambda = 0,02$, la imagen resultante me dio un mínimo de 0 y un máximo aproximado de 0.9814.

3.6. Pruebas básicas

Hice algunas pruebas sencillas modificando el número de iteraciones y el valor de ϵ . El objetivo fue revisar el comportamiento básico del código, sin realizar todavía un ajuste fino de los parámetros.

3.6.1. Número de iteraciones

Mantuve fijos:

$$\epsilon = 0,1, \quad \lambda = 0,02 \quad (22)$$

Y probé corriendo 5, 10 y 30 iteraciones.

Cuadro 4: Resultado al cambiar el número de iteraciones.

Iteraciones	Mínimo	Máximo
5	0.0000	0.9986
10	0.0000	0.9814
30	0.0019	0.8074

Como se observa en la tabla, al aumentar el número de iteraciones el efecto de la difusión se hace más visible. Con 5 iteraciones el máximo se mantiene prácticamente en 1, pero al llegar a 30 iteraciones el valor máximo baja hasta aproximadamente 0.81.

Visualmente también noté que el borde del cuadrado se va difuminando más a medida que avanza el proceso.

3.6.2. Efecto de epsilon

También quise comparar el efecto de ϵ , así que probé dos valores distintos manteniendo fijos $\lambda = 0,02$ y 10 iteraciones.

Cuadro 5: Resultados para distintos valores de ϵ .

ϵ	$c_{\min}$	$c_{\max}$	$\lambda_{\max}$	Máx. resultado
0.1	1.4003	10.0	0.025	0.9814
0.2	1.3608	5.0	0.050	0.9949

Al aumentar ϵ de 0.1 a 0.2, el valor máximo de c baja a la mitad (de 10 a 5), lo que a su vez permite un límite mayor para λ .

En este caso, al usar el mismo λ y el mismo número de iteraciones, observé que con $\epsilon = 0,1$ la imagen queda un poco más suavizada. Esto es consistente con el comportamiento esperado, ya que el coeficiente de difusión puede alcanzar valores mayores cuando ϵ es más pequeño.

En resumen, estas pruebas iniciales me confirmaron que tanto ϵ como el número de iteraciones tienen un impacto directo en el resultado, aunque creo que necesitaría evaluar más valores para sacar conclusiones definitivas.

3.7. Limitaciones

Por falta de tiempo no alcancé a implementar las dos propuestas adicionales de coeficientes de difusión ni la comparación final entre los métodos.

Tampoco logré hacer el estudio completo de parámetros ni las pruebas finales sobre la imagen real que pedía el enunciado. Por esta razón, los resultados de esta sección corresponden más que nada a la validación de que mi implementación base para el modelo TV funciona de forma correcta.